

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота 2
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
« Розробка графічного редактора об'єктів »

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки новий проект з ім'ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

Індивідуальні параметри варіанту (для номера в журналі Ж = 3)

- **Структура масиву об'єктів:** Динамічний масив вказівників Shape `**pcshape`.
- **Розмір масиву:** 103 елементи.
- **"Гумовий" слід при вводі:** Пунктирна лінія чорного кольору.
- **Прямокутник:**
 - *Спосіб вводу:* Від центру до одного з кутів.
 - *Відображення:* Чорний контур прямокутника без заповнення. Колір заповнення ігнорується, оскільки фігура прозора.
- **Еліпс:**
 - *Спосіб вводу:* По двом протилежним кутам охоплюючого прямокутника.
 - *Відображення:* Чорний контур із кольоровим заповненням.
 - *Колір заповнення:* Блакитний.
- **Позначка поточного типу об'єкту:** У заголовку вікна.

Вихідний текст файлів:

Lab2.cpp

```
#include "framework.h"
#include "Lab2.h"
#include "shape.h"

#define MAX_LOADSTRING 100
const int MY_ZH = 3;
// Обсяг динамічного масиву = 103 (згідно Ж=3)
const int MAX_SHAPES = MY_ZH + 100;

HINSTANCE hInst;
WCHAR szTitle[MAX_LOADSTRING];
WCHAR szWindowClass[MAX_LOADSTRING];

// Динамічний масив поліморфних вказівників (варіант Ж mod 3 = 0)
Shape** pcshape = nullptr;
int shapeCount = 0;

bool isDrawing = false;
long startX, startY, endX, endY;

int currentTool = 0; // 0-Крапка, 1-Лінія, 2-Прямокутник, 3-Еліпс
Shape* tempShape = nullptr; // Вказівник для "гумового" сліду

ATOM        MyRegisterClass(HINSTANCE hInstance);
BOOL        InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void        UpdateWindowTitle(HWND hWnd);

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPWSTR lpCmdLine,
_In_ int nCmdShow) {
    LoadStringW(hInstance, IDS_APP_TITLE, szTitle, MAX_LOADSTRING);
    LoadStringW(hInstance, IDC_LAB2, szWindowClass, MAX_LOADSTRING);

    // Виділяємо пам'ять під динамічний масив масивів (на 103 елементи)
    pcshape = new Shape * [MAX_SHAPES];

    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;
    HACCEL hAccelTable = LoadAccelerators(hInstance, MAKEINTRESOURCE(IDC_LAB2));
    MSG msg;

    while (GetMessage(&msg, nullptr, 0, 0)) {
        if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg)) {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }

    // Очищення динамічної пам'яті перед закриттям програми
    for (int i = 0; i < shapeCount; i++) delete pcshape[i];
    delete[] pcshape;
```

```
    return (int)msg.wParam;
}
```

```
ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(hInstance, MAKEINTRESOURCE(IDI_LAB2));
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB2);
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(IDI_SMALL));
    return RegisterClassExW(&wcex);
}
```

```
BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);
    if (!hWnd) return FALSE;

    UpdateWindowTitle(hWnd); // Встановлюємо стартовий заголовок
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}
```

```
Shape* CreateShape(int tool) {
    switch (tool) {
        case 0: return new PointShape();
        case 1: return new LineShape();
        case 2: return new RectShape();
        case 3: return new EllipseShape();
    }
    return nullptr;
}
```

```
void UpdateWindowTitle(HWND hWnd) {
    LPCWSTR toolNames[] = { L"Крпка", L"Лінія", L"Прямокутник", L"Еліпс" };
    WCHAR newTitle[256];
    wprintfW(newTitle, L"Lab2 - %s", toolNames[currentTool]);
    SetWindowTextW(hWnd, newTitle);
}
```

```
HPEN hRubberPen = nullptr;
```

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
```

```

case WM_COMMAND: {
    int wParam = LOWORD(wParam);
    HMENU hMenu = GetMenu(hWnd);

    CheckMenuItem(hMenu, ID_OBJECT_POINT, MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_LINE, MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_RECT, MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_ELLIPSE, MF_UNCHECKED);

    switch (wParam) {
        case ID_OBJECT_POINT: currentTool = 0; CheckMenuItem(hMenu, ID_OBJECT_POINT, MF_CHECKED); break;
        case ID_OBJECT_LINE:  currentTool = 1; CheckMenuItem(hMenu, ID_OBJECT_LINE, MF_CHECKED); break;
        case ID_OBJECT_RECT:  currentTool = 2; CheckMenuItem(hMenu, ID_OBJECT_RECT, MF_CHECKED); break;
        case ID_OBJECT_ELLIPSE: currentTool = 3; CheckMenuItem(hMenu, ID_OBJECT_ELLIPSE, MF_CHECKED); break;
        case IDM_EXIT:        DestroyWindow(hWnd); break;
        default: return DefWindowProc(hWnd, message, wParam, lParam);
    }
    UpdateWindowTitle(hWnd);
    break;
}

case WM_LBUTTONDOWN:
    if (shapeCount < MAX_SHAPES) {
        isDrawing = true;
        startX = endX = LOWORD(lParam);
        startY = endY = HIWORD(lParam);
        tempShape = CreateShape(currentTool);

        hRubberPen = CreatePen(PS_DASH, 1, RGB(0, 0, 0));
    }
    break;

case WM_MOUSEMOVE:
    if (isDrawing && tempShape != nullptr) {
        HDC hdc = GetDC(hWnd);

        SetROP2(hdc, R2_NOTXORPEN);
        SetBkMode(hdc, TRANSPARENT);

        HPEN oldPen = (HPEN)SelectObject(hdc, hRubberPen);

        tempShape->Set(startX, startY, endX, endY);
        tempShape->Show(hdc);

        endX = LOWORD(lParam);
        endY = HIWORD(lParam);

        tempShape->Set(startX, startY, endX, endY);
        tempShape->Show(hdc);

        SelectObject(hdc, oldPen);
    }
}

```

```

        ReleaseDC(hWnd, hdc);
    }
    break;

case WM_LBUTTONDOWN:
    if (isDrawing) {
        isDrawing = false;

        pcshape[shapeCount] = CreateShape(currentTool);
        pcshape[shapeCount]->Set(startX, startY, endX, endY);
        shapeCount++;

        delete tempShape;
        tempShape = nullptr;

        if (hRubberPen != nullptr) {
            DeleteObject(hRubberPen);
            hRubberPen = nullptr;
        }

        InvalidateRect(hWnd, NULL, TRUE);
    }
    break;

case WM_PAINT: {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);

    // ПОЛИМОРФИЗМ:
    for (int i = 0; i < shapeCount; i++) {
        pcshape[i]->Show(hdc);
    }

    EndPaint(hWnd, &ps);
    break;
}

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

```

shape.cpp

```

#include "shape.h"

void Shape::Set(long x1, long y1, long x2, long y2) {

```

```

    xs1 = x1; ys1 = y1;
    xs2 = x2; ys2 = y2;
}

void PointShape::Show(HDC hdc) {
    SetPixel(hdc, xs2, ys2, RGB(0, 0, 0));
}

void LineShape::Show(HDC hdc) {
    MoveToEx(hdc, xs1, ys1, NULL);
    LineTo(hdc, xs2, ys2);
}

void RectShape::Show(HDC hdc) {
    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH));

    long left = xs1 - (xs2 - xs1);
    long top = ys1 - (ys2 - ys1);

    Rectangle(hdc, left, top, xs2, ys2);

    SelectObject(hdc, oldBrush);
}

void EllipseShape::Show(HDC hdc) {
    HBRUSH fillBrush = CreateSolidBrush(RGB(0, 255, 255));
    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc, fillBrush);

    Ellipse(hdc, xs1, ys1, xs2, ys2);

    SelectObject(hdc, oldBrush);
    DeleteObject(fillBrush);
}

```

shape.h

```

#pragma once
#include <windows.h>

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
public:
    virtual ~Shape() {}

    void Set(long x1, long y1, long x2, long y2);

    virtual void Show(HDC hdc) = 0;
};

class PointShape : public Shape {

```

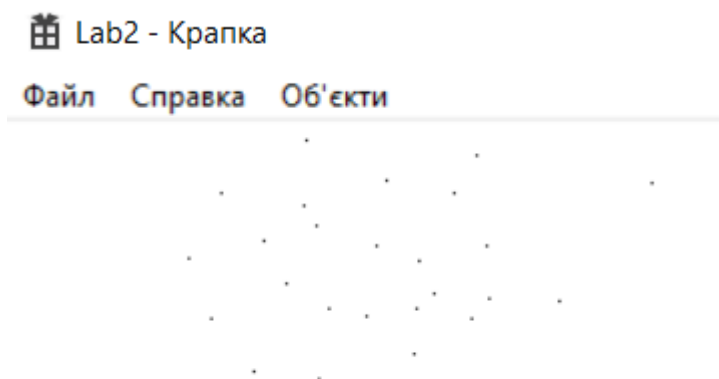
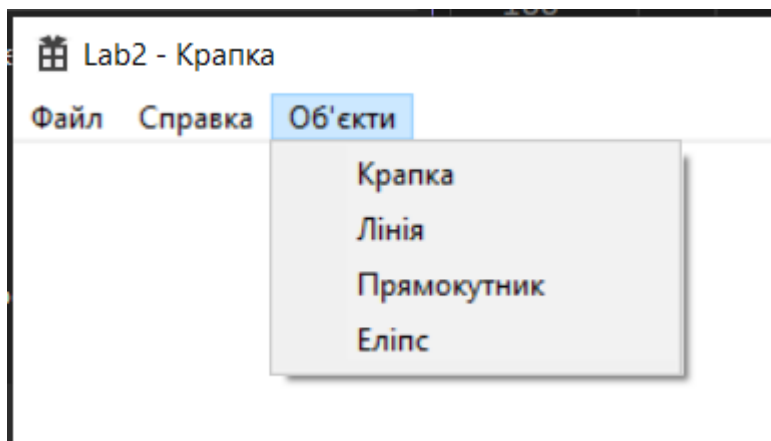
```
public:
    void Show(HDC hdc) override;
};
```

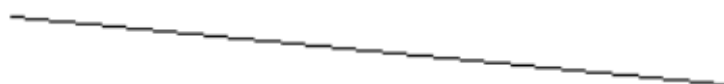
```
class LineShape : public Shape {
public:
    void Show(HDC hdc) override;
};
```

```
class RectShape : public Shape {
public:
    void Show(HDC hdc) override;
};
```

```
class EllipseShape : public Shape {
public:
    void Show(HDC hdc) override;
};
```

Скріншоти:





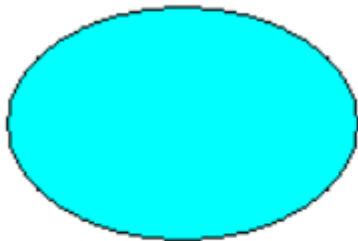
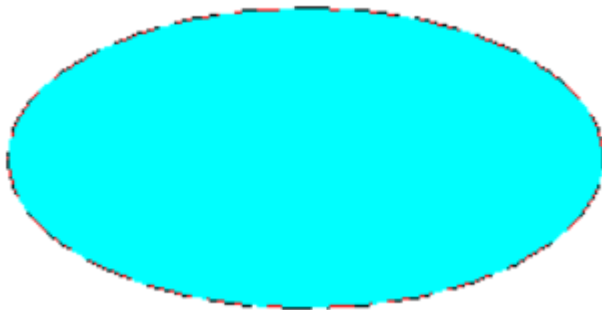
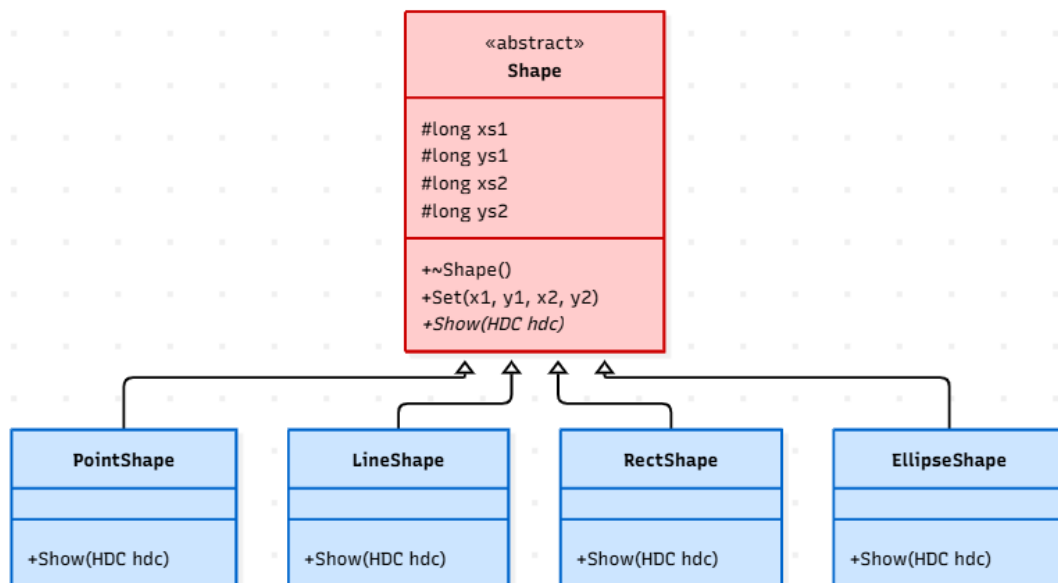


Схема успадкування класів:



Висновки:

Я отримав вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об'єктно-орієнтованому стилі.